

题目提供的 **C-- 文法** (无 `while/for` 循环, 无数组, 但支持递归、`if-else`、全局变量、`float`) , 我设计了 4 个具有针对性的复杂测试用例。

这些用例主要测试：**递归逻辑** (代替循环)、**深层 `if-else` 嵌套** (测试移进-归约冲突)、**复杂算术优先级**以及**作用域/函数调用**。

测试用例 1：递归斐波那契数列 (Recursion & Arithmetic)

这个用例测试函数定义、递归调用以及基础的算术运算。由于没有循环，递归是实现重复计算的唯一方式。

```
// test_complex_1.sy
int fib(int n) {
    if (n <= 1) {
        return n;
    }
    // 测试复杂的递归调用和加法
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int N = 10;
    int result = 0;
    result = fib(N);
    return result;
}
```

测试用例 2：悬空 Else 与 复杂逻辑运算 (Dangling Else & Logic)

这个用例专门用来“折磨”语法分析器，测试 `if` 嵌套匹配 (最近匹配原则) 以及 `&&` 和 `||` 的优先级。

这个大概率是有问题的，复杂逻辑运算能力很弱，中间代码如果test6过不去就别测这个了。主要是由于老师提供的`compile_ir`基本不支持这些复杂的运算，但是可以尝试一下，不行的话就截止到语法解析就可以了。

```
// test_complex_2.sy
int main() {
    int a = 10, b = 20, c = 30;
    int res = 0;

    // 测试悬空 Else (Dangling Else)
    ! // 这里的 else 应该匹配第二个 if (b < c)
    if (a < b)
        if (b < c)
            res = 1;
        else
            res = 2;

    // 测试逻辑运算符优先级: && 应该高于 ||
    ! // 逻辑应解析为: (a < b) || ((b > c) && (a == 10))
    if (a < b || b > c && a == 10) {
```

```

        res = res + 100;
    }

    // 深度嵌套与块作用域
    if (res > 0) {
        int deep = 5;
        if (deep != 0) {
            res = res * 2;
        }
    } else {
        res = 0;
    }

    return res;
}

```

测试用例 3：最大公约数与浮点数混合 (GCD & Float)

测试多个函数、取模运算 `%`（这在优先级中通常与乘除同级）以及浮点数常量的解析。

这个大概率也过不去，中间代码如果test7过不去就别测这个了。

```

// test_complex_3.sy
float PI = 3.14159; // 全局浮点常量

int gcd(int a, int b) {
    if (b == 0) return a;
    // 测试取模运算
    return gcd(b, a % b);
}

int main() {
    int x = 24, y = 18;
    int common = gcd(x, y); // 应该是 6

    // 浮点数运算
    float r = 5.0;
    float area = 0.0;

    if (common > 1) {
        // 测试混合类型赋值（如果编译器支持隐式转换）
        // 或者单纯测试复杂的乘法连算
        area = PI * r * r;
    }

    return 0;
}

```

测试用例 4：终极表达式压力测试 (Expression Stress Test)

这个用例没有复杂的控制流，全是括号和运算符，用来测试 `Exp` -> `AddExp` -> `MulExp` 等层级是否能正确处理极其复杂的数学公式。

这个大概率也有问题，中间代码如果test8过不去就别测这个了

```
// test_complex_4.sy
int foo(int val) {
    return val * 2;
}

int main() {
    int a = 5;
    int b = 10;
    int c = 2;
    int result = 0;

    // 疯狂的优先级测试 能不能v我50可恶的软工
    // 应该解析为: a + ( (b * c) / (a - 2) ) - ( (foo(a) % 3) * b )
    // 包含: 函数调用、括号嵌套、四则运算混合
    result = a + b * c / (a - 2) - foo(a) % 3 * b;

    // 关系运算作为值 (如果你的语言支持布尔值转0/1)
    // x = ( (a > b) == 0 )
    int logic_val = (a > b) == 0;

    return result;
}
```